From an image to an invoice

Proposed approach: read the image, check it, then use Fakturama's UI.

I would use a small Python program with separate extraction, validation, desktop and workflow functions. One order image supplies the data for an Order and its linked Invoice. Separating image reading from desktop actions makes the business rules easier to test and explain.



Unclear or inconsistent? Stop before using the UI.

Reading the image

I would start with local Tesseract OCR, which reads printed text and records each word's position. Labels and table headings would group the words into customer details, addresses, payment data and item rows. The image, raw text and field crops would stay with the result so mistakes can be traced back.

Local OCR needs no paid API and is easy to inspect, but it needs layout rules for the printed form. An optional AI extractor could return the same data model and pass the same checks; it would be a chosen alternative, rather than an automatic fallback.

Checking the data

Required fields, dates, names and totals must agree before desktop work starts. For the sample, Marta Klein becomes first name Marta and last name Klein. Ambiguous names need review. Product codes (SKUs), leading zeroes, item order and repeated SKUs must be preserved.

Use decimal arithmetic (Python Decimal) to avoid floating-point errors in money. Check line, net, VAT and gross totals against the image. Probe Fakturama's rounding ties and VAT aggregation before accepting those cases; unexplained differences stop the run.

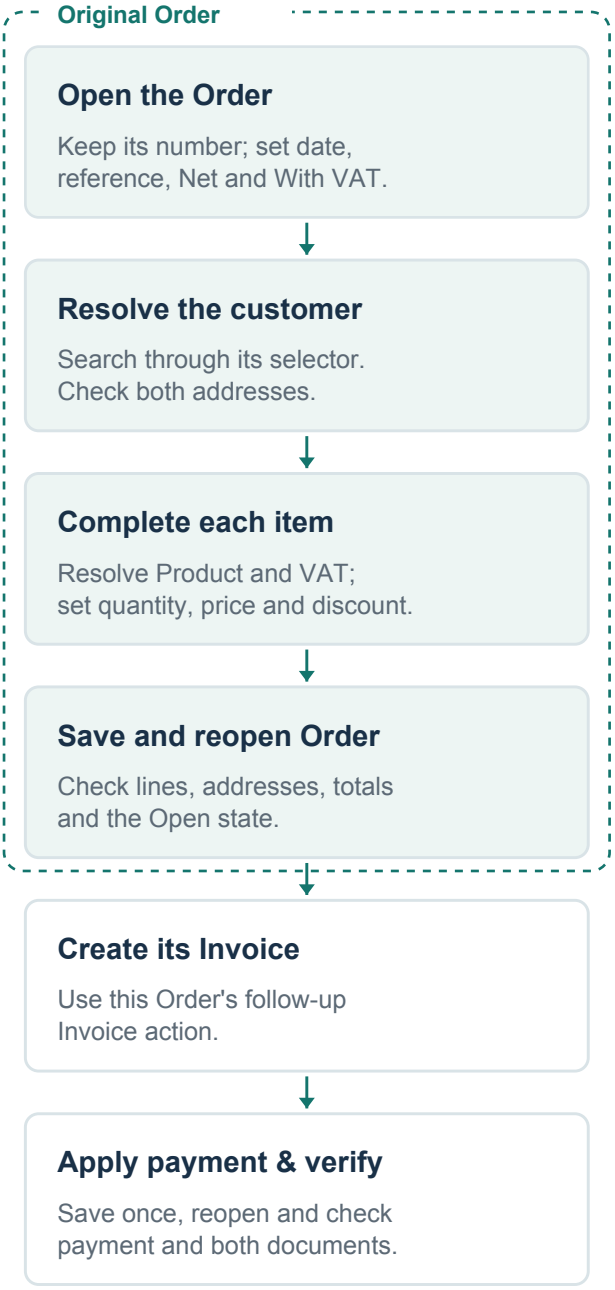
Two prices with different purposes

Order line: 2 chairs x EUR 250.00 x 90% = **EUR 450.00 net**.
Reusable Product: EUR 250.00 x 1.19 = **EUR 297.50 gross per chair**.
The 10% Order discount does not change the reusable Product price.

The same Order stays open

Resolve missing records inside the Order-first workflow.

Keep Fakturama's proposed record numbers. Set the image's Order date and reference, choose Net and With VAT, then keep this Order open throughout customer and Product creation.



Search before creating

Use the Order's selectors. A customer matches by company, first and last name, ZIP and city; a Product matches by full SKU. Select one matching row. Create only when readable results establish absence; conflicts or ambiguity need review.

Customer and payment

Create a Debtor with billing as Invoice address. Reuse that address for Delivery when identical; otherwise add a Delivery address with its recipient. If payment is missing, keep the Debtor editor open, resolve it through Data > terms of payment, then return and select it. Save the Debtor once and reselect it through the Order.

Products and VAT

For a missing Product, resolve VAT first by name, percentage and S (Standard rate) code. Save a new VAT once, then create the Product with undiscounted gross price and zero cost/stock. Save once and reselect it. Every Order line receives its own quantity, net price, VAT and discount.

Payment codes

Bank Transfer: Credit transfer
Credit Card: Credit card
SEPA Direct Debit: SEPA direct debit

New methods use zero discounts and days, blank account/text fields and no standard-setting change.

Use the Order's follow-up Invoice action and keep the proposed Invoice number, date and service date. For PAID, enter the image's payment date and full Invoice total. Otherwise leave Paid clear without inventing a date or value. The original Order must remain Open.

Find, act and check

Verify each step before continuing; stop when the outcome is unclear.

Locating controls

I would combine OCR of labels and table headings with matching of Fakturama's selector icons. On macOS, accessibility can supplement this with exposed control types, values and actions. OCR covers visible controls that are missing from the accessibility tree; native values can avoid OCR mistakes when reading editable text.

Each action would check the app, active editor, window bounds and screen scaling, then locate a unique target in that area. Table cells would use current headers and the intended item row. After input, commit the edit and read it back within a bounded timeout. Reobserve after layout changes rather than retaining screen coordinates.

Checking the results

Before Save, record the intended action. Send Save once, retrieve the record through its selector or Documents, and reopen it to compare values. Final checks cover both addresses, all item rows, totals, payment and the Order-Invoice relationship. Matching references alone are insufficient to establish that relationship.

If a control, Save or follow-up outcome is uncertain, stop with screenshots and a report of expected and observed values. A simple sequential workflow is easier to inspect than a recovery engine. Automatic replay or rollback could duplicate or undo an action that already completed.

Build and validate in stages

Use English Fakturama on macOS with isolated test data. Before transactions, check dependencies, permissions, currency, document settings and the required columns. Prioritize the supplied form and a continuous workflow before broader layouts or optional AI extraction.

1. Extraction and arithmetic. Compare every field and row with the image. Test missing or ambiguous text, repeated SKUs, mismatched totals and rounding. These tests need no desktop writes.

2. Record selection and persistence. Probe matching, missing and conflicting customers, Products, VAT and payment definitions. Check email and both address roles after reopening; test all three payment mappings and selection from the still-open Debtor editor.

3. Complete Order and Invoice. Run from the original image in fresh data. Verify every line, warehouse recipient and address association, copied Invoice fields, paid and unpaid states, and the retained Open Order. Then exercise moved/resized windows and uncertain outcomes.